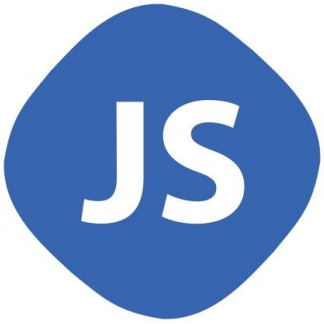


Обработка ошибок

- Введение
- Исключения
- Объект ошибки
- Генерация собственных ошибок
- Проброс исключения
- try…catch…finally

Введение

Неважно, насколько мы хороши в программировании, иногда наши скрипты содержат ошибки. Они могут возникать из-за наших промахов, неожиданного ввода пользователя, неправильного ответа сервера и по тысяче других причин.

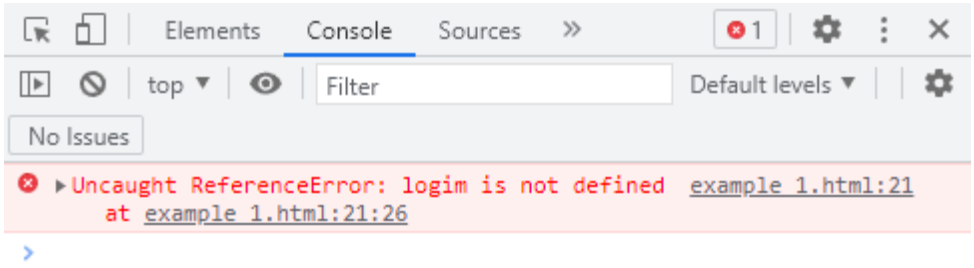
Обычно скрипт в случае ошибки *падает* (сразу же останавливается), с выводом ошибки в **консоль**.

example_1. Ошибка в коде - обычное дело

```
<script>
	// объявляем переменные
	let name, login;
	// инициируем переменные
	name = "Карим";
	login = "admin";
	// ошибка, переменная -> logim <- не определена
	let user = `${name} (${logim})`;
</script>
```



В скрипте произошла банальная опечатка в имени переменной и скрипт *упал*. В консоли указана причина остановки скрипта.



Но есть синтаксическая конструкция **try...catch**, которая позволяет *ловить* ошибки и вместо *падения* делать что-то более осмысленное.

Синтаксис try...catch

Конструкция **try...catch** состоит из двух основных блоков: **try**, и затем **catch**, но может использоваться только с одним, **обязательным** блоком **try**.

```
try {
     try_statements
     // основной код
}[catch(err) {
     catch_statements
     // обработка ошибки
}][finally {
     finally_statements
     // завершающий код
}]
```

Параметры

- **try_statements** - код для выполнения и контроля над генерируемыми им исключениями.
- **catch_statements** - все исключения будут попадать в блок catch_statements.

- **err** - переменная, которой присваивается пойманное исключение. Переменной можно давать любое имя.
- **finally_statements** - блок finally_statements будет выполнен после окончания работы try...catch, вне зависимости от того, было ли сгенерировано исключение

> ***К сведению***. Некоторые ошибки иногда называют **исключениями** (какие и когда читаем дальше).

Если детали ошибки вас не интересуют, то аргумент при вызове **catch** можно пропустить:

```
try{
    // ...
}catch{ // <- без (err)
    // ...
}
```

Описание

Конструкция **try...catch** в JavaScript может быть представлена в своей полной форме, включая необязательную секцию **finally**.

Допускается использование всех трех форм:

- **try...catch**
- **try...finally**
- **try...catch...finally**

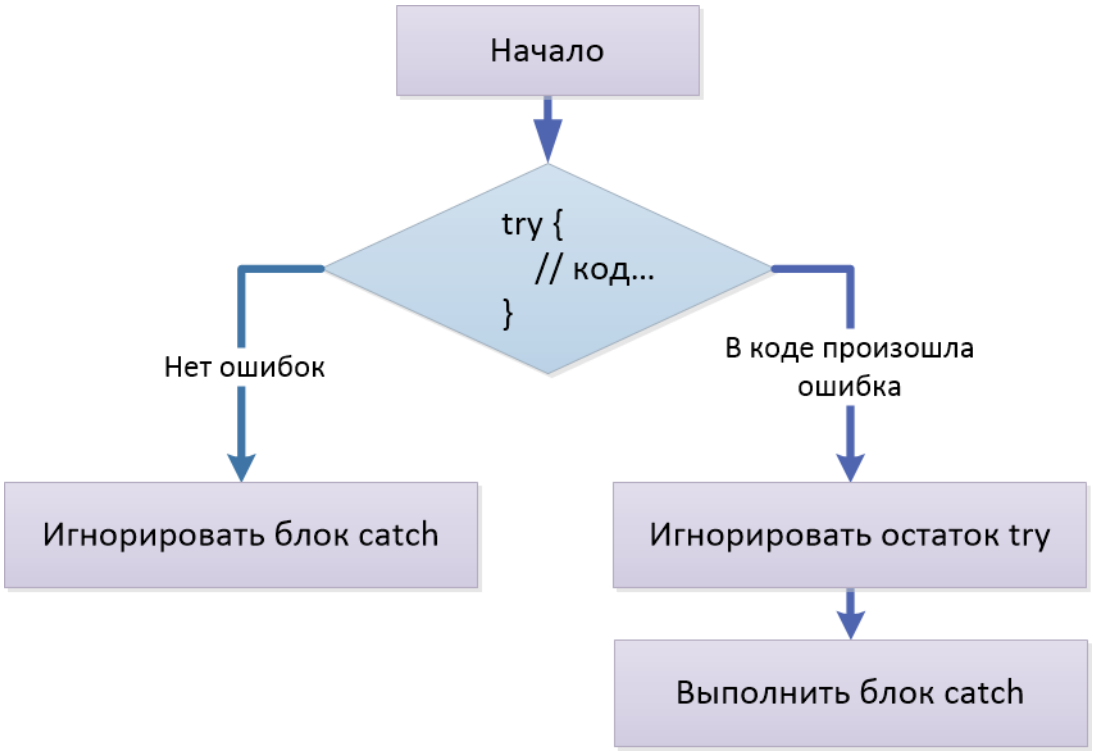
Работает конструкция следующим образом:

1. Сначала выполняется код внутри блока **try** {...}.
2. Если в нём нет ошибок, то блок **catch**(**err**) игнорируется: выполнение доходит до конца **try** и потом далее, полностью пропуская **catch**.
3. Если же в нём возникает ошибка, то выполнение **try** прерывается, и поток управления переходит в начало **catch**(**err**). Переменная **err**

(можно использовать любое имя) содержит объект ошибки с подробной информацией о произошедшем.

> ***Важно***. Таким образом, при ошибке в блоке **try** {…} скрипт не *падает*, и мы получаем возможность обработать ошибку внутри **catch**.

Схематично конструкцию **try**…**catch** можно представить следующим образом.



Рассмотрим примеры.

В примере **example_2** выполнение кода блока **try** прерывается, и **поток управления** переходит в начало блока **catch**.

example_2. Генерация ошибки в блоке try

```
<script>
    try {
        // объявляем переменные
        let name, login;
        // инициируем переменные
```

```
        name = "Карим";
        login = "admin";
        // ошибка, переменная -> logim <- не определена
        let user = `${name} (${logim})`;
    } catch {
        // сообщение о возникшем исключении
        alert(`Возникла ошибка!`);
    }
</script>
```

В следующем примере последние две инструкции блока **try** никогда не выполнятся (ну, то есть до тех пор, пока не будет исправлена опечатка ;)

example_3. Размещение инструкции в блоке try не гарантия выполнения

```
<script>
    try {
        console.log ('Начало блока try');
        // объявляем переменные
        let name, login;
        // ошибка, переменная -> logim <- не определена
        let user = `${name} (${logim})`;
        console.log ('Конец блока try (никогда не
        выполнится)');
        alert ("Как жалко, что мы никогда не встретимся...");
    } catch {
        // сообщение о возникшем исключении
        alert(`Возникла ошибка!`);
    }
```

```
</script>
```

Исключения

try...catch работает только для ошибок, возникающих во время выполнения кода. Чтобы try...catch работал, код должен быть выполнимым. Другими словами, это должен быть **корректный JavaScript-код**.

Он не сработает, если код **синтаксически неверен**, например, содержит несовпадающее количество открывающих скобок:

```
// JS движок не прочитает такой файл,
// файл синтаксически некорректен
try {
     console.log (('В коде присутствует лишняя скобка');
     // ...
} catch {
     // ...
}
```

JavaScript-движок сначала **читает код**, а затем **исполняет его**.

> ***Важно***. Ошибки, которые возникают во время фазы **чтения кода**, называются **ошибками парсинга**.

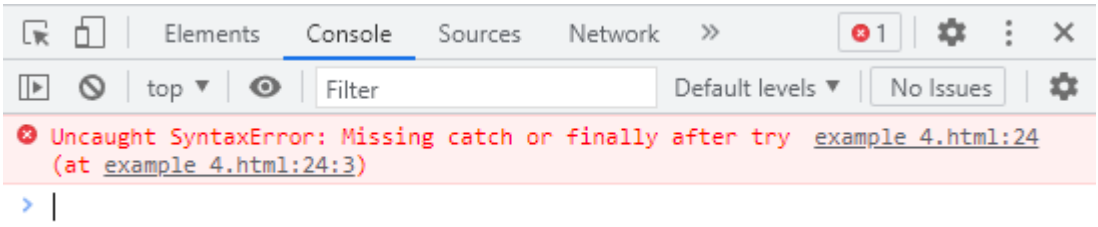
Их нельзя обработать (изнутри этого кода), потому что движок не понимает код.

В демонстрационном примере **example_4** все таже неопределенная переменная (**logim**), но код в секции **catch** не сработает, возникла ошибка парсинга.

example_4. Пример ошибки парсинга

```
<script>
    try {
        console.log ('Начало блока try');
        // объявляем переменные
        let name, login;
          // ошибка, переменная -> logim <- не определена
        let user = `${name} (${logim})`;
        console.log ('Конец блока try');
    // ошибка, опечатка в ключевом слове catch
    } cath {
        // сообщение о возникшем исключении не возникнет
        alert(`Возникла ошибка!`);
    }
</script>
```

Исключения не возникло. Возникла синтаксическая ошибка.



Таким образом, **try...catch** может обрабатывать только ошибки, которые возникают в корректном коде. Такие ошибки называют ошибками времени выполнения.

Важно. Ошибки, которые возникают во время фазы **исполнения кода**, называют **ошибками времени выполнения**, а иногда **исключениями**.



Объект ошибки

Когда возникает ошибка, JavaScript **генерирует объект ошибки**, содержащий её детали. Затем этот объект передаётся как аргумент в блок **catch**:

```
try {
     // ...
} catch(err) { // объект ошибки, можно использовать любое имя
     // ...
}
```

Для всех встроенных ошибок объект имеет два основных свойства:

- **name** - имя ошибки. Например, для неопределённой переменной это ReferenceError.
- **message -** текстовое сообщение о деталях ошибки.

В большинстве окружений доступны и другие, нестандартные свойства. Одно из самых широко используемых и поддерживаемых – это:

- **stack** - текущий стек вызова: строка, содержащая информацию о последовательности вложенных вызовов, которые привели к ошибке. Используется в целях отладки.

Тестируем работу с объектом ошибки. Обратите внимание, при выводе объекта ошибки в консоль, тип **object** будет приведен к **строке**.

example_5. Выводим в консоль информацию об ошибке

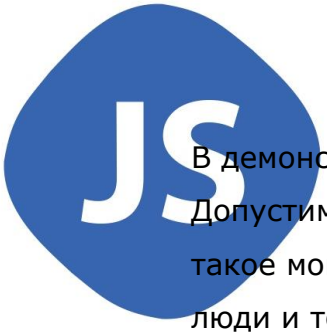
```
<script>
    try {
        let name, login;
        // место генерирования объекта ошибки
        let user = `${name} (${logim})`;
```

```
        // имя может быть любым
    } catch(err) {
        console.log("Тип ошибки: ", typeof err); // object
        // можем вывести ошибку
        // ошибка типа object приводится к строке
        console.log("Сообщение: ", err); // ReferenceError: logim
        is not defined
    }
</script>
```

В следующем примере выводим **свойства** объекта ошибки.

example_6. Свойства объекта ошибки

```
<script>
    try {
        let name, login;
        // место генерирования объекта ошибки
        let user = `${name} (${logim})`;
    } catch(error) {
        console.log("Свойства ошибки:");
        console.log("Свойство name: ", error.name); //
        ReferenceError
        console.log("Свойство message: ", error.message); //
        logim is not defined
        console.log("Свойство stack: ", error.stack); //
        ReferenceError: logim is not defined
    }
</script>
```



В демонстрационном примере **example_7** моделируем еще одну ситуацию. Допустим, скрипт получает **некорректные данные JSON формата**. Почему такое могло случиться, тема отдельного разговора, но вообще бэкендеры тоже люди и тоже совершают ошибки ;)

JavaScript поддерживает метод **JSON.parse**() для чтения JSON данных. Обычно он используется для декодирования данных, полученных по сети, от сервера или из другого источника.

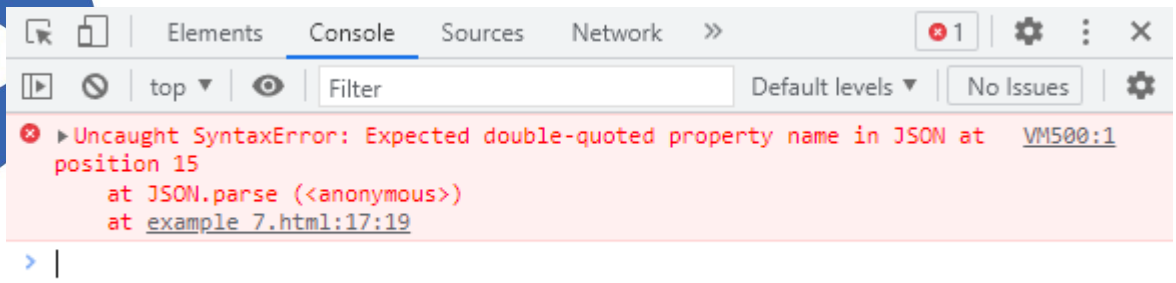
Вызываем метод **JSON.parse**() для чтения и преобразования данных в JS-объект.

example_7. Некорректный формат JSON данных

```
<script>
    // такие данные могли прийти с сервера
    // свойство age не обернуто кавычками
    let json = '{"name":"John",age:30}';
    // преобразуем текстовое JSON-представление в JS-объект
    let user = JSON.parse(json);
    // хотим получить JS-объект user со свойствами name и age
    document.write(user.name); // John
    document.write(user.age);  // 30
</script>
```

Если JSON некорректен, **JSON.parse** генерирует ошибку, то есть скрипт *падает*.

Пренебрежение конструкцией обработки ошибок, дает *рухнувший* скрипт с выводом сообщения синтаксической ошибки (SyntaxError) в консоль (плюс потеря вами репутации адекватного программиста*).



P.S. * - репутацию адекватного человека вы потеряли, когда решили стать программистом ... (или нет?).

Не думаю, что такое поведение скрипта устроит пользователя. Получается, что если вдруг что-то не так с данными, то пользователь никогда (если, конечно, не откроет консоль) об этом не узнает. А пользователи очень не любят, когда что-то просто *падает* без всякого сообщения об ошибке.

Скрипты, особенно скрипты, работоспособность которых зависит не только от вас, например скрипты обмена данными с другими приложениями, обязательно должны сопровождаться **конструкциями анализа** и **обработки кода** на предмет появления ошибок.

example_8. Скрипты обмена данными требуют тщательного анализа кода

```
<script>

    try {

        // такие данные могли прийти с сервера

        let json = '{"name":"John",age:30}';

        // преобразуем текстовое JSON-представление в JS-объект

        let user = JSON.parse(json);

        // user - JS-объект со свойствами name и age

        document.write(user.name); // John

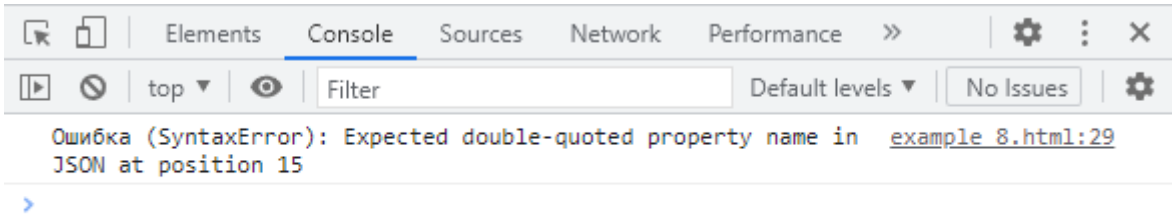
        document.write("<br>");

        document.write(user.age);  // 30

    } catch(err){
```

```
        // возможно, не имеет смысла посвящать во все тонкости
        пользователя, поэтому ему достаточно и такого сообщения
        alert( "Извините, при получении данных произошла
        ошибка.\nВиновные будут приговорены к тюремным срокам!");
        // а вот веб-разработчику куда полезнее окажется эта
        информация
        console.log ("Ошибка (" + err.name + "): " +
        err.message);
    }
</script>
```

Здесь мы используем блок **catch** только для вывода сообщения, но разработчик может сделать гораздо больше: отправить новый сетевой запрос, предложить попробовать получить данные позже, обозвать пользователя неудачником ... *



Любое действие лучше, чем просто *падение*.

Ваши приложения будут выглядеть лучше, если вы помните, что мир несовершенен, жизнь неидеальна, а программный код пишут люди.

P.S. * - про неудачника это шутка была ...

Генерация собственных ошибок

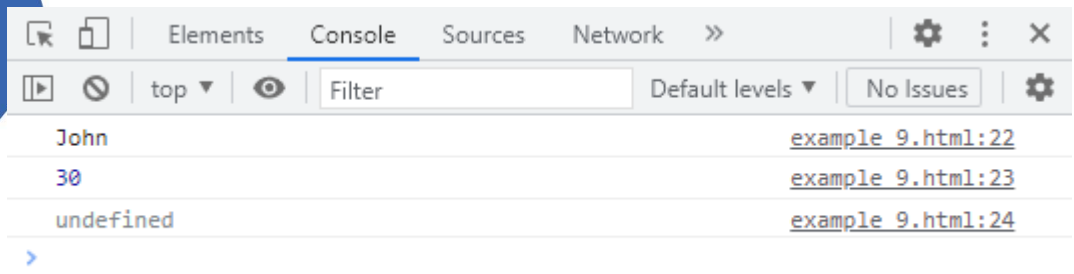
Но что, если JSON синтаксически корректен, но не содержит необходимого нам свойства **login** (или любого другого)?

Например, так:

example_9. Случай, когда хотелось бы получить ошибку

```
<script>
    try {
        // данные с сервера
        let json = '{"name":"John","age":30}';
        // преобразуем текстовое JSON-представление в JS-объект
        let user = JSON.parse(json);
        // user - JS-объект со свойствами - name и age
        // и отсутствующим свойством - login
        console.log(user.name); // John
        console.log(user.age);  // 30
        console.log(user.login);  // ???
    } catch(err){
        // хорошо бы получить ошибку, но синтаксически все верно
        и ошибки не будет
        console.log ("Ошибка (" + err.name + "): " +
        err.message);
    }
</script>
```

Здесь **JSON.parse** выполнится без ошибок, но на самом деле отсутствие свойства **login** для нашего приложения ошибка.



В таких случаях широко используется практика самостоятельного **создания** и **генерирования** объекта ошибки. Для этих целей в JavaScript есть множество встроенных конструкторов стандартных ошибок: **Error**, **SyntaxError**, **ReferenceError**, **TypeError** и другие.

В демонстрационном примере **example_10** приведу несколько примеров самостоятельного создания **встроенных ошибок** некоторых типов.

example_10. Создание объектов ошибок

```
<script>

    // ошибка типа Error

    let error = new Error("Шеф, все пропало, все пропало!!! Гипс
    снимают, клиент уезжает!");

    console.log(error.name); // Error

    console.log(error.message); // Шеф, все пропало ...

    // ошибка типа SyntaxError

    let syntaxError = new SyntaxError("Все на выход. Сохраняйте
    спокойствие. Или паникуйте и кричите, уже не важно.");

    console.log(syntaxError.name); // SyntaxError

    console.log(syntaxError.message); //  Все на выход.
    Сохраняйте спокойствие ...

    // ошибка типа ReferenceError

    let referenceError = new ReferenceError("Приказ избегать
    паники отменяется! Пусть все сидят по домам и боятся!");
```

```
    console.log(referenceError.name); // ReferenceError
    console.log(referenceError.message); // Приказ избегать
    паники отменяется ...
</script>
```

Как видим, создавать ошибки очень легко (в отличии от способности писать код без них ;)

> ***К сведению***. Конструктор объектов - новое понятие в курсе. С объектами поработаем в специально отведенной для этих целей теме.

Ошибку мало создать, ее необходимо вызвать (генерировать). Для того чтобы **вызвать объект ошибки**, воспользуемся оператором **throw**.

Оператор throw

Оператор **throw** генерирует ошибку указанного типа.

Синтаксис

```
throw <объект_ошибки>
```

Генерировать ошибку можно, например, так, как показано в примере **example_11**.

example_11. Создание и вызов ошибки

```
<script>
    // создаем объект ошибки типа Error
    let error = new Error("Шеф, все пропало, все пропало!!! Гипс
    снимают, клиент уезжает!");
    // генерируем ошибку
    throw error;
    // чтобы увидеть это сообщение закомментируйте вызов throw
```

```
    console.log("Не будет выведено, в случае генерирования
    ошибки");
</script>
```

Код может быть короче, если **совместить** процесс **создания** и **генерирования** ошибки, например так:

```
// создаем и вызываем объект ошибки типа Error
throw new Error("Шеф, все пропало, все пропало!!! Гипс снимают, клиент уезжает!");
// или так
// создаем и вызываем объект ошибки типа SyntaxError
throw new SyntaxError("Все на выход. Сохраняйте спокойствие. Или паникуйте и кричите, уже не важно.");
```

Для встроенных ошибок (не для любых объектов, только для ошибок):

- свойство **name** – имя конструктора,
- свойство **message** – значение аргумента.

Например:

```
let myError = new ReferenceError("Приказ избегать паники отменяется! Пусть все сидят по домам и боятся!");
// myError.name – ReferenceError
// myError.message – "Приказ избегать паники отменяется! Пусть все сидят по домам и боятся!"
```

Как итог, создадим сценарий с вызовом и обработкой ошибки отсутствия свойства **login** в полученных данных.

example_12. Обработка ошибки отсутствия свойства login пересылаемых данных

```
<script>
```

```
    try {
        // данные с сервера
        let json = '{"name":"John","age":30}';
        // преобразуем текстовое JSON-представление в JS-объект
        let user = JSON.parse(json);
        // user - JS-объект со свойствами - name и age
        console.log("Начинаем выводить свойства объекта:");
        console.log(user.name); // John
        console.log(user.age);  // 30
        // свойство login может отсутствовать
        // поэтому проверяем его особо
        if (user.login){
            console.log(user.login);
        } else {
        // генерируем ошибку типа Error со свойством message
        // переадресуем выполнение в блок catch
            throw Error ("Упс. Что-то пошло не так...");
    }
    } catch(err){
        // ситуация, когда ошибка скорее хорошо, чем плохо
        // выводим информацию о сгенерированной ошибке
        console.log (err.name + ". ", err.message);
        console.log ("Ошибка обработки JSON данных.");
    }
</script>
```

Технически в качестве объекта ошибки можно передать что угодно. Это может быть **пользовательский объект** или даже примитив: **число** или **строка**, но всё же лучше, чтобы это был объект, желательно со свойствами **name** и **message** (для совместимости со встроенными ошибками).

example_13. Вместо объекта стандартной ошибки - строка

```
<script>
    try {
    if (0){
        // ...
    } else {
        // вместо объекта стандартной ошибки - строка
        throw "Упс. Что-то пошло не так ...";
    }
    } catch(err){
        // вывод сообщения ошибки
        console.log(err);
}
</script>
```

example_14. Вместо объекта стандартной ошибки - число

```
<script>
    try {
        if (0){
        // ...
    } else {
        // вместо объекта стандартной ошибки - число
```

```
        throw 555;
    }
    } catch(err){
        // вывод информации об ошибки
        console.log(err);
    }
</script>
```

example_15. Вместо объекта стандартной ошибки – пользовательский объект

```
<script>
    // пользовательский объект ошибки
    let objErr = {
        name: "UserError",
        message: "Упс. Что-то пошло не так ...",
        description: "Пользовательский объект ошибки"
    }
    try {
        if (0){
        // ...
    } else {
        // вместо объекта стандартной ошибки - пользовательский
        объект
        throw objErr;
    }
    } catch(err){
        // вывод информации об ошибке
```

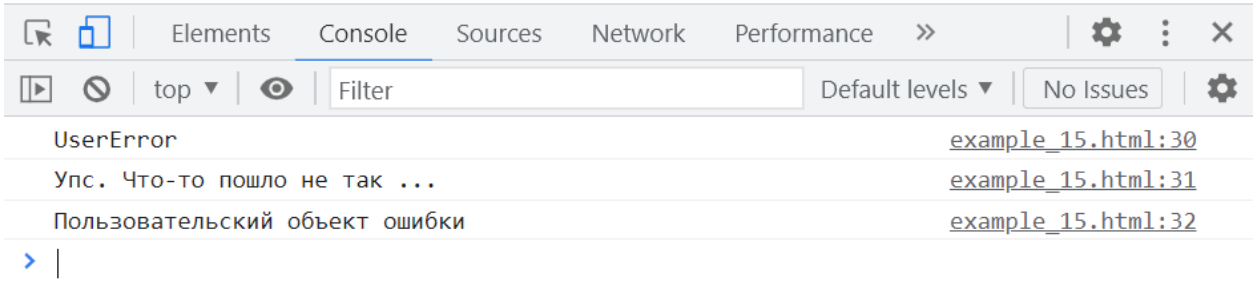
```
        console.log(err.name);

        console.log(err.message);

        console.log(err.description);

    }
</script>
```

Пример вывода свойств пользовательского объекта ошибки в консоль.



Проброс исключений

В примерах выше мы использовали конструкцию **try...catch** для обработки ошибок, происхождение которых нам было хорошо известно.

А что, если в блоке **try** {...} возникнет **другая неожиданная** ошибка? Например, программная (неопределённая переменная) или какая-то ещё.

```
<script>
    try {
        // данные с сервера
        let json = '{"name":"John","age": 30}';

        // возможная ошибка преобразования данных
        let user = JSON.parse(json);

        // возможная ошибка отсутствия данных
        if (!user.login){
            throw ReferenceError ("Неполные данные сервера");
        }

        /*
            код, в котором возможно появление других ошибок
        */
```

Конечно, возможно все! Программисты совершают ошибки. Даже в утилитах с открытым исходным кодом, используемых миллионами людей на протяжении десятилетий – вдруг может быть обнаружена ошибка, которая приводит к ужасным взломам.

По своей природе **catch** получает *все* свои ошибки из **try**. И если он получает неожиданную ошибку, но показывает заранее подготовленное сообщение, например: "Ошибка парсинга JSON строки", то такое поведение неправильно и только **затрудняет отладку кода**.

К счастью, мы можем выяснить, какую ошибку мы получили, например, по её свойству **name**. При этом надо понимать, что синтаксическая (SyntaxError) ошибка может появиться не только вследствие преобразования JSON-строки, также как и ошибка – ReferenceError результат не только отсутствующего свойства login.

> ***К сведению.*** Крайне аккуратно относитесь к сопровождающему ошибки тексту.

example_16. Техника проброса исключений

```
<script>

    try {

        // данные с сервера

        let json = '{"name":"John","age": 30}';

        // возможная ошибка преобразования данных

        let user = JSON.parse(json);

        // возможная ошибка отсутствия данных

        if (!user.login){

            throw ReferenceError ("Неполные данные сервера");

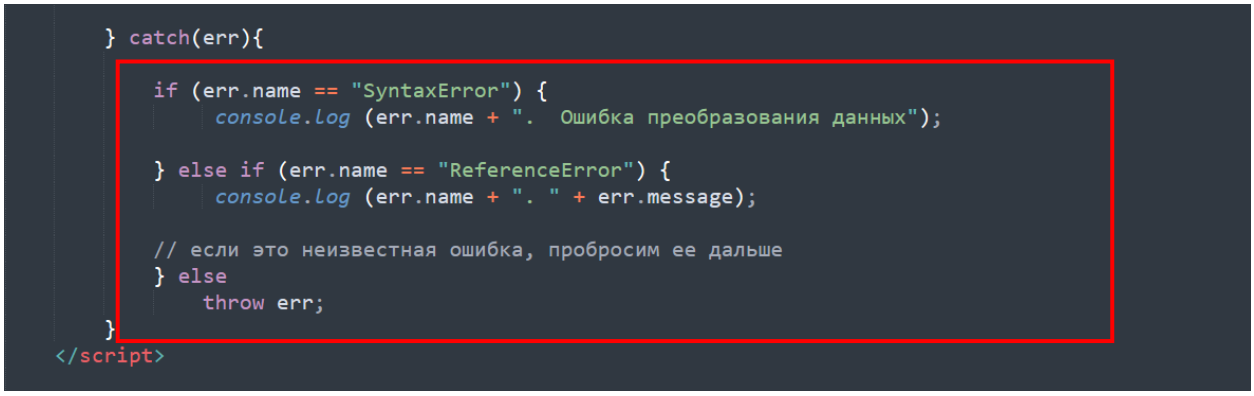
        }

        /*
```

```
        код, в котором возможно появление других ошибок
        */
    } catch(err){
        if (err.name == "SyntaxError") {
             console.log (err.name + ".  Ошибка преобразования
            данных");
        } else if (err.name == "ReferenceError") {
             console.log (err.name + ". " + err.message);
        // если это неизвестная ошибка, пробросим ее дальше
        } else
            throw err;
    }
</script>
```

В коде примера example_16 секция **catch** обрабатывает только те ошибки, происхождение которых нам **известно**: **SyntaxError** или **ReferenceError**.

Ошибки, происхождение которых нам **неизвестно**, пробрасываем дальше, такая техника обработки ошибок называется **пробросом исключения**.



Важно. Есть простое правило, блок **catch** должен обрабатывать только те ошибки, которые ему известны, и **пробрасывать** все остальные.

Техника **проброс исключения** выглядит так:

- Блок **catch** получает все ошибки.
- В блоке **catch**(err) {...} мы анализируем объект ошибки **err**.
- Если мы не знаем, как её обработать, тогда делаем **throw err**.

Проброшенная из блока **catch** ошибка **выпадает наружу** и может быть поймана другой внешней конструкцией try...catch (если таковая имеется), или *убьёт* скрипт.

> ***Важно.*** Таким образом, блок **catch** фактически обрабатывает только те ошибки, с которыми он знает, как справляться, и пропускает остальные.

> ***К сведению.*** В теме, посвященной **функциям**, рассмотрим, как такие ошибки могут быть пойманы с помощью ещё одного уровня **try...catch**.

try...catch...finally

Конструкция **try...catch** может содержать ещё одну секцию: **finally**.

Если секция есть, то она **выполняется в любом случае**:

- после **try**, если не было ошибок
- после **catch**, если ошибки были.

Логику работы конструкции при этом можно описать следующей схемой:

```
try {
     // попытка выполнить основной код
}[catch(err) {
     // обработка ошибки в случае возникновения
}][finally {
     // код, который выполняется всегда
```



В следующем демонстрационном примере секция **finally** будет выполняться всегда, вне зависимости от решения пользователя на генерацию ошибки.

example_17. Секция finally

```
<script>
    try {
        // программа проявляет чудеса толерантности
        if (confirm("Вы хотите получить ошибку?")){
            // ну, если пользователь так хочет ...
            throw Error("Ошибка на ровном месте... :-(");
        } else {
            // жить без ошибок - здорово!
            alert ("Спасибо, вы настоящий друг!");
        }
    } catch(err){
        // вывод информации об ошибке
        console.log("Пользователь тешится ;)");
        console.log(err.name + ": " + err.message);
        // следующий блок выполнится в любом случае
    } finally {
        // красивые стихи уместны всегда
        let msg = `
        <pre><h2>
        О сколько нам открытий чудных
        Готовят просвещенья дух
        И Опыт, сын ошибок трудных,
```



```
            И Гений, парадоксов друг,
            И Случай, бог изобретатель.</h2>
            (Александр Пушкин)
            </pre>`;
            // вывод сообщения в браузер
            document.write(msg);
        }
</script>
```

У кода есть два пути выполнения:

- Если вы **утвердительно** ответите на вопрос: "Вы хотите получить ошибку?", то **try -> catch -> finally**.
- Если ответите **отрицательно**, то **try -> finally**.

Секцию **finally** часто используют, когда мы начали что-то делать и хотим завершить это вне зависимости от того, будет ошибка или нет.

> ***К сведению***. Секция **finally** отлично подходит для завершения действий сетевого обмена данными.

В **самостоятельных работах** к этому уроку:

- Поиск и устранение синтаксических ошибок формирования JSON строки. Вывод JSON-данных с помощью вложенных циклов.
- Создание и генерация пользовательских объектов ошибок.
- Нахождение корней квадратного уравнения с генерацией ошибки.
- Генерация ошибки по решению пользователя.